

Maestría en
Software

Patrones de Diseño de Software

Autor(es): Mauricio Ortiz Ochoa
mortizo@ups.edu.ec

Tema:

Unidad 03. Patrones de diseño de comportamiento

Tema:

State

- *State*
- Observer

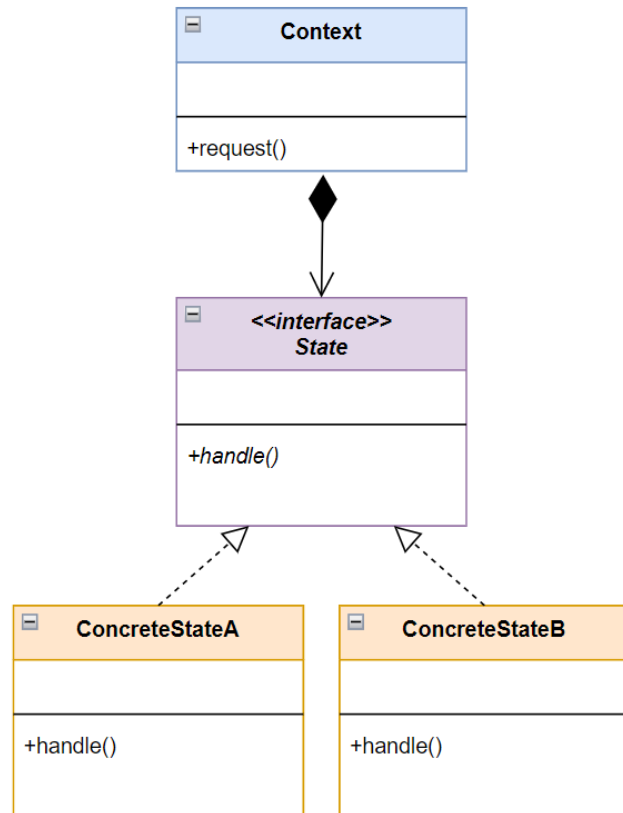
State

State

- Propósito
 - Permite que un objeto modifique su comportamiento cuando cambia su estado interno.
 - Desde el punto de vista del cliente, parecerá que el objeto cambia su forma de actuar.
- Problema / Motivación
 - Supongamos que en una Universidad una solicitud de certificado electrónico puede encontrarse en tres estados:
Iniciado
En Trámite
Firmado
 - Las mismas acciones sobre la solicitud pueden producir resultados diferentes según su estado:
Iniciado: se puede generar el documento, pero todavía no visualizarlo ni descargarlo.
En Trámite: se puede visualizar con marca de agua y descargar una versión provisional.
Firmado: se puede visualizar y descargar el documento final.
 - Si cada operación debe verificar continuamente el estado mediante múltiples if o switch, la lógica comienza a dispersarse y cada nuevo estado obliga a modificar varias operaciones.
 - Necesitamos que la solicitud pueda cambiar su comportamiento a medida que cambia su estado, sin concentrar todas esas decisiones en una sola clase.

State

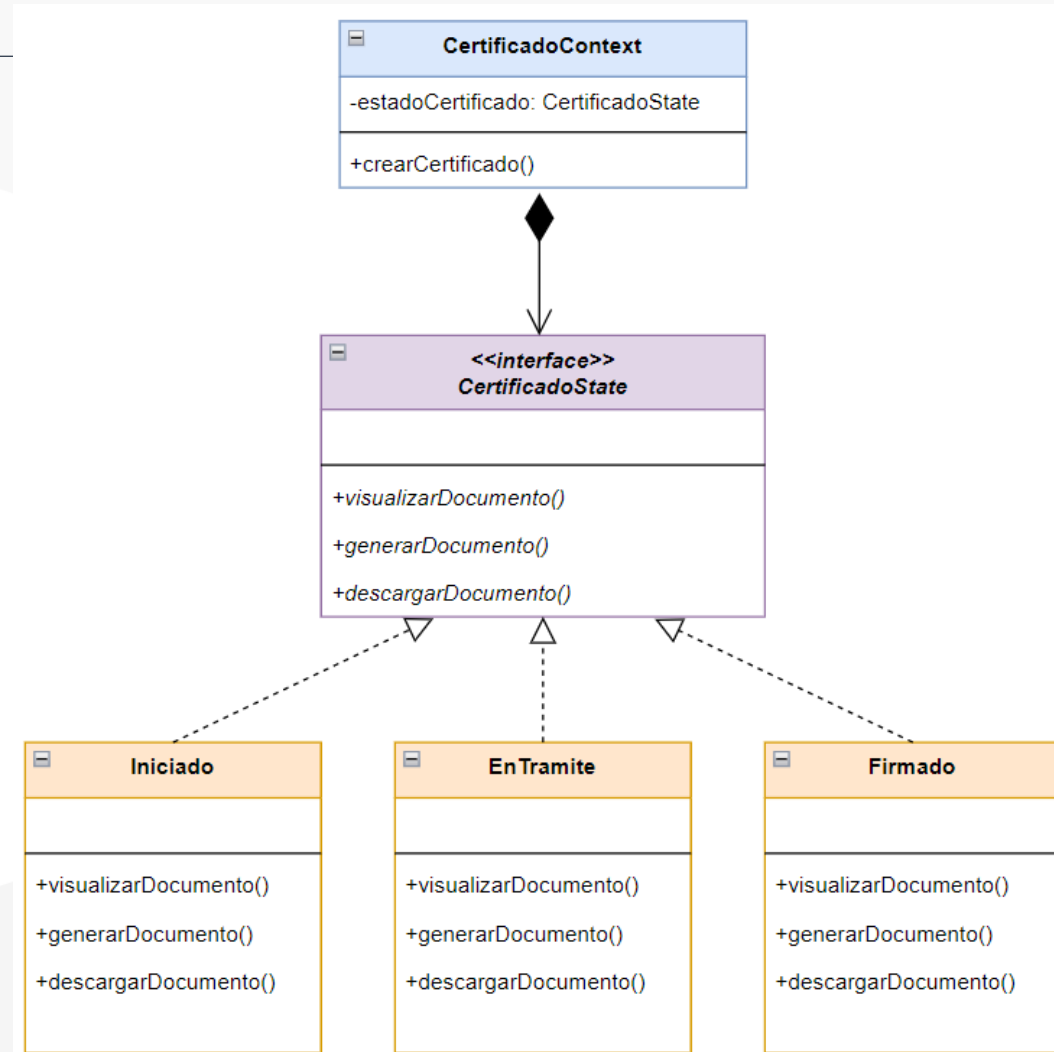
Estructura



- Participantes
 - **Context**
Define la interfaz utilizada por los clientes.
Mantiene una referencia al objeto State que representa su estado actual.
Delega en ese estado las operaciones cuyo comportamiento depende de la situación actual.
 - **State**
Define una interfaz común para encapsular el comportamiento asociado a los diferentes estados del Context.
 - **ConcreteState:**
Implementa el comportamiento específico correspondiente a un estado concreto del Context.
- Colaboraciones
 - Context delega las operaciones dependientes del estado en el objeto State actual.
 - Un State puede recibir una referencia al Context cuando necesita consultar información o provocar una transición.
 - Los clientes interactúan principalmente con el Context, no con los estados concretos.
 - La transición hacia un nuevo estado puede ser controlada por el Context o por un ConcreteState, según las responsabilidades definidas en el diseño.

State

Solución



<https://github.com/mortizo/pds-unidad03-ooms/tree/master/pds-unidad03-ooms/src/main/java/p69/state>

State

State

- Aplicabilidad
 - El comportamiento de un objeto dependa de su estado interno y deba cambiar dinámicamente durante su ciclo de vida.
 - Varias operaciones contengan condicionales repetidos que dependen del mismo estado.
 - Existan transiciones relevantes entre estados y cada estado tenga responsabilidades claramente diferenciadas.
 - Incorporar un nuevo estado obligue a modificar múltiples operaciones de la clase principal.
- Evite utilizarlo cuando:
 - El objeto tenga pocos estados y el comportamiento asociado sea simple.
 - Un enum acompañado de un condicional claro resuelva suficientemente el problema.
 - Las transiciones sean mínimas o poco probables de cambiar.
 - Crear una clase por estado introduzca más complejidad que beneficio.
- Consecuencias / Riesgos
 - Incrementa el número de clases.
 - Las transiciones pueden quedar dispersas si no se define claramente quién las controla.
 - Puede dificultar el seguimiento del flujo si existen demasiados estados y transiciones.

State

Actividad en clase: ¿State?

Caso 1 — Pedido en una tienda en línea

Un pedido puede encontrarse en los estados Creado, Pagado, Enviado y Entregado.

Las operaciones disponibles cambian según el estado: cancelar, pagar, modificar dirección, consultar guía o confirmar recepción.

¿Cómo diseñarían el sistema para que el pedido cambie su comportamiento durante su ciclo de vida sin acumular condicionales en cada operación?

Caso 2 — Ticket de soporte

Un ticket puede estar Abierto, Asignado, En atención, Resuelto o Cerrado.

Acciones como reasignar, responder, reabrir o cerrar dependen del estado actual del ticket.

¿Cómo distribuirían el comportamiento y las transiciones para evitar que cada operación tenga que verificar continuamente el estado?

Caso 3 — Préstamo bibliotecario

Un préstamo puede encontrarse Activo, Vencido, Devuelto o Bloqueado.

Renovar, devolver o aplicar una sanción produce resultados diferentes dependiendo del estado actual.

¿Cómo diseñarían el préstamo para que las mismas operaciones respondan de forma distinta según su estado interno?

Caso 4 — Reproductor multimedia

Un reproductor puede estar Detenido, Reproduciendo o Pausado.

Las acciones play, pause y stop deben comportarse de manera diferente dependiendo del estado actual.

¿Cómo diseñarían el reproductor para encapsular el comportamiento de cada estado y gestionar sus transiciones?

Tema:

Observer

- State
- *Observer*

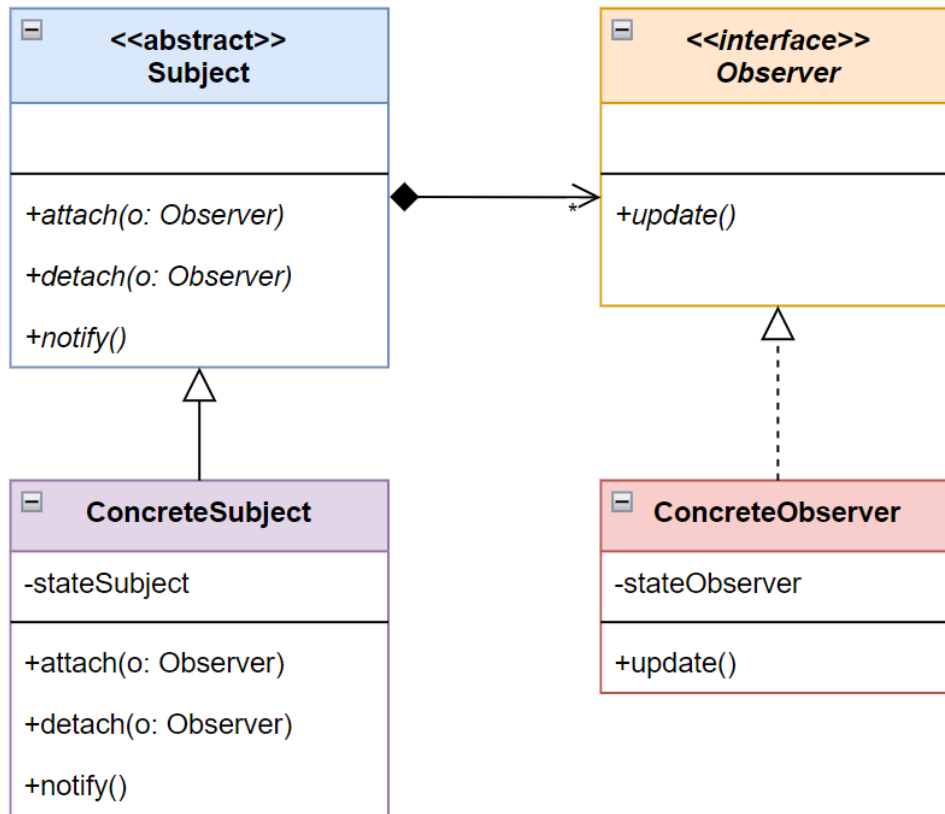
Observer

Observer

- Propósito
 - Define una dependencia uno-a-muchos entre objetos, de manera que cuando un objeto cambia de estado, todos los objetos interesados son notificados y pueden reaccionar al cambio.
- Problema / Motivación
 - Supongamos que una empresa procesa pedidos en línea y genera una factura asociada a cada pedido.
 - Cuando una factura cambia al estado Pagada, diferentes áreas deben ejecutar acciones:
 - Despacho: inicia el proceso de envío.
 - Servicio al Cliente: inicia el seguimiento y posteriormente puede enviar una encuesta de satisfacción.
 - Contabilidad: registra los asientos contables correspondientes.
 - Si Factura debe conocer y llamar directamente a cada una de estas áreas, cada nuevo interesado obliga a modificarla y aumenta el acoplamiento.
 - Necesitamos que distintos objetos puedan suscribirse y reaccionar a los cambios de la factura, sin que esta conozca sus implementaciones concretas.

Observer

Estructura



Participantes

- **Subject**

Mantiene una colección de objetos Observer.
Proporciona operaciones para suscribir y desuscribir observadores.
Define el mecanismo para notificarlos cuando ocurre un cambio relevante.

- **ConcreteSubject**

Mantiene el estado de interés para los observadores.
Notifica a los observadores cuando dicho estado cambia.

- **Observer**

Define una interfaz común para los objetos que deben ser notificados ante cambios en el Subject.

- **ConcreteObserver**

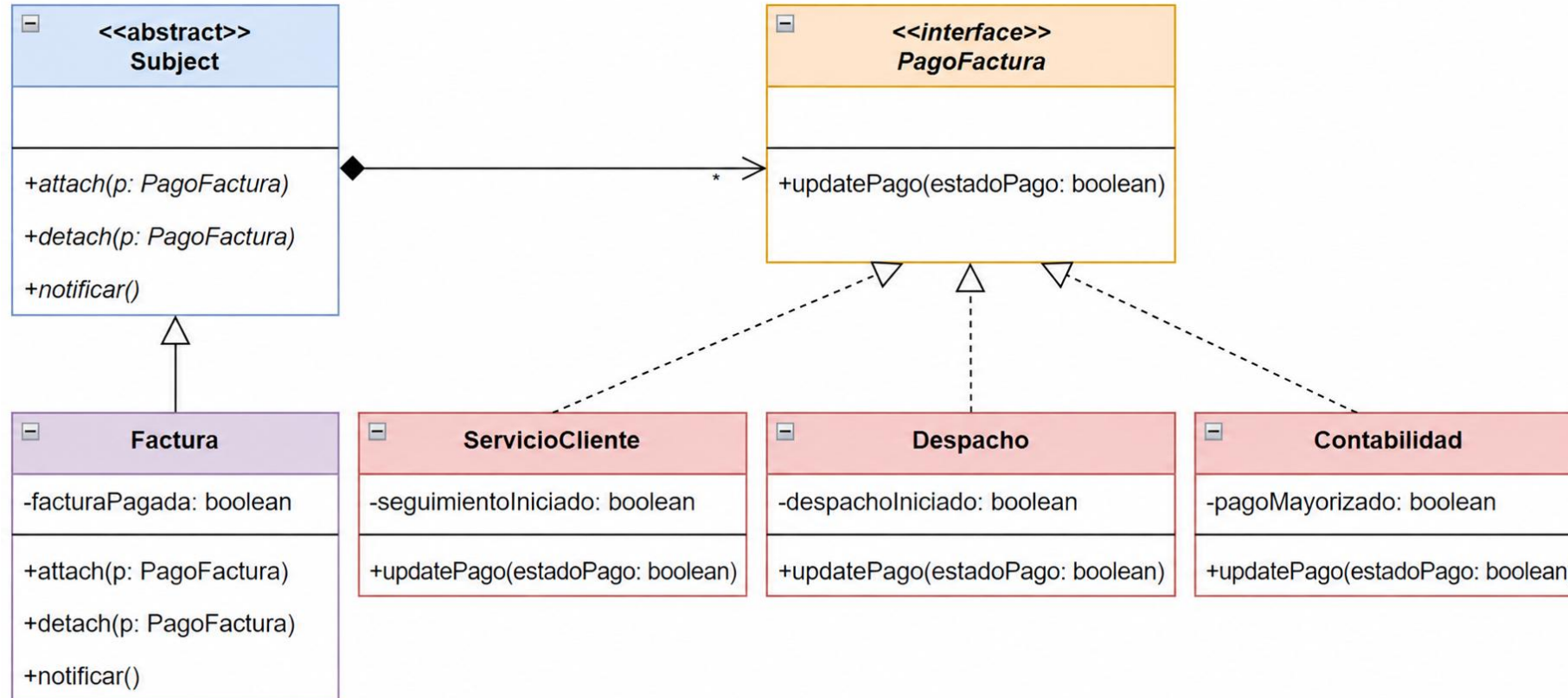
Implementa la operación de actualización definida por Observer.
Puede mantener una referencia al ConcreteSubject cuando necesita consultar información adicional.
Reacciona al cambio manteniendo su información consistente con la del sujeto.

Colaboraciones

- ConcreteSubject notifica a los observadores registrados cuando ocurre un cambio relevante.
- Cada ConcreteObserver recibe la notificación mediante la interfaz Observer.
- Después de ser notificado, un observador puede consultar al Subject para obtener la información necesaria y reaccionar al cambio.

Observer

Solución



<https://github.com/mortizo/pds-unidad03-ooms/tree/master/pds-unidad03-ooms/src/main/java/p69/observer>

Observer

Observer

- Aplicabilidad
 - Un cambio en un objeto deba provocar reacciones en varios objetos dependientes.
 - No se conozca de antemano cuántos objetos necesitarán reaccionar.
 - El objeto que origina el cambio no deba depender de las clases concretas que reaccionan.
 - Los observadores puedan agregarse o eliminarse dinámicamente.
 - Se necesite mantener bajo acoplamiento entre el emisor del cambio y los receptores.
- Evite utilizarlo cuando:
 - Exista un único receptor conocido y estable.
 - Una llamada directa resulte más simple y clara.
 - El orden de las notificaciones sea crítico y difícil de controlar.
 - Las notificaciones puedan provocar cadenas difíciles de seguir.
 - La indirección introduzca más complejidad que beneficio.
- Consecuencias / Riesgos
 - Puede dificultar el seguimiento del flujo de ejecución.
 - Puede generar notificaciones en cascada.
 - El orden de actualización de los observadores puede no ser evidente.
 - Los observadores deben gestionarse correctamente para evitar referencias innecesarias o suscripciones olvidadas.

Observer

Actividad en clase: ¿Observer?

Caso 1 — Disponibilidad de productos

Una tienda en línea permite que varios clientes indiquen interés en productos temporalmente agotados. Cuando un producto vuelve a estar disponible, deben ser informados los clientes interesados.

¿Cómo diseñarían el sistema para notificar a los interesados sin acoplar el producto a cada cliente concreto?

Caso 2 — Publicación de calificaciones

Cuando un docente publica una calificación, diferentes componentes pueden necesitar reaccionar: el estudiante debe recibir una notificación, el sistema académico actualizar estadísticas y otro servicio registrar el cambio.

¿Cómo permitirían agregar o eliminar componentes interesados sin modificar continuamente el objeto que publica la calificación?

Caso 3 — Monitoreo de sensores

Un sensor registra cambios de temperatura. Diferentes componentes pueden reaccionar ante una nueva medición: una pantalla actualiza el valor, un sistema de alertas verifica umbrales y un módulo histórico almacena el registro.

¿Cómo diseñarían la comunicación para que el sensor no dependa directamente de todos los componentes que reaccionan?

Caso 4 — Subasta electrónica

Una subasta recibe nuevas ofertas durante su ejecución. Diversos participantes necesitan enterarse cuando cambia la oferta más alta para actualizar su información o decidir si realizan una nueva oferta.

¿Cómo diseñarían el sistema para que nuevos participantes puedan incorporarse o retirarse sin modificar la lógica de la subasta?

Unidad 03.02

Retroalimentación y cierre

Qué vimos hoy

- State encapsula el comportamiento asociado al estado actual de un objeto y permite que cambie durante su ciclo de vida.
- Observer permite que varios objetos reaccionen ante un cambio sin acoplar el emisor a sus implementaciones concretas.
- En ambos patrones, el objetivo no es añadir clases, sino distribuir mejor responsabilidades e interacciones.

Antes de finalizar:

¿El comportamiento cambia porque cambió la situación interna del objeto? → State

¿Un cambio debe propagarse a varios objetos interesados? → Observer

¿Quién controla las transiciones entre estados?

¿Quién origina el cambio y quiénes deben reaccionar?

¿Realmente necesitamos el patrón o una solución más simple sería suficiente?

Idea final: State distribuye el comportamiento según el estado actual; Observer distribuye la reacción a un cambio entre múltiples interesados

Cierre del curso

Diseñar bien no es perder tiempo; es ganar futuro

- Los patrones de diseño no son recetas para complicar el software.
- Son una forma de pensar con orden, criterio y disciplina.
- Nos ayudan a construir soluciones más claras, mantenibles y preparadas para el cambio.
- A veces aplicar un patrón parece tomar más tiempo al inicio, pero en realidad ahorra retrabajo, errores y frustraciones en el futuro.
- En el desarrollo de software, como en la vida, el orden y la disciplina no siempre se ven de inmediato, pero siempre dan frutos.
- Diseñar mejor hoy es evitar problemas mañana.

En software, como en la vida, el orden, la disciplina y las buenas decisiones al inicio evitan muchos problemas después.

Cierre del curso

DISEÑAR BIEN ES GANAR FUTURO

Hacer un buen código es un acto de responsabilidad con nuestros compañeros. No basta con que funcione: también debemos procurar que los demás encuentren en nuestro código **aprecio, claridad y confianza**, no confusión ni desorden.

“El servicio a los demás es el arriendo que pagamos por vivir en este mundo.”
— Muhammad Ali

Hoy invertimos tiempo en aprender.

Mañana evitamos errores.

Construimos software **claro y sostenible**.

Servimos mejor a los demás.

¡Sigan adelante!

El buen diseño, el orden y la disciplina siempre dan frutos.

Anexos

Bibliografía

Gamma, E., Helm, R., Johnson, R. y Vlissides, J.; Design Patterns: Elements of Reusable Object-Oriented Software; 1.ª edición, 1994.

Sarcar, V.; Java Design Patterns: A Hands-On Experience with Real-World Examples; 3.ª edición, 2022.

Musch, O.; Design Patterns with Java: An Introduction; 1.ª edición, 2023.

Fowler, M.; UML Distilled: A Brief Guide to the Standard Object Modeling Language; 3.ª edición, 2004.

Shvets, A.; Sumérgete en los patrones de diseño; 1.ª edición, 2019.

IEEE Computer Society; Guide to the Software Engineering Body of Knowledge — SWEBOK Guide V4.0a; versión 4.0a, 2025.

Asaad, J. y Avksentieva, E.; Assessing the Impact of GoF Design Patterns on Software Engineering Practices; 2025.

Gracias por su atención

posgrados.ups.edu.ec